



Cloud-Native Computing and Microservices

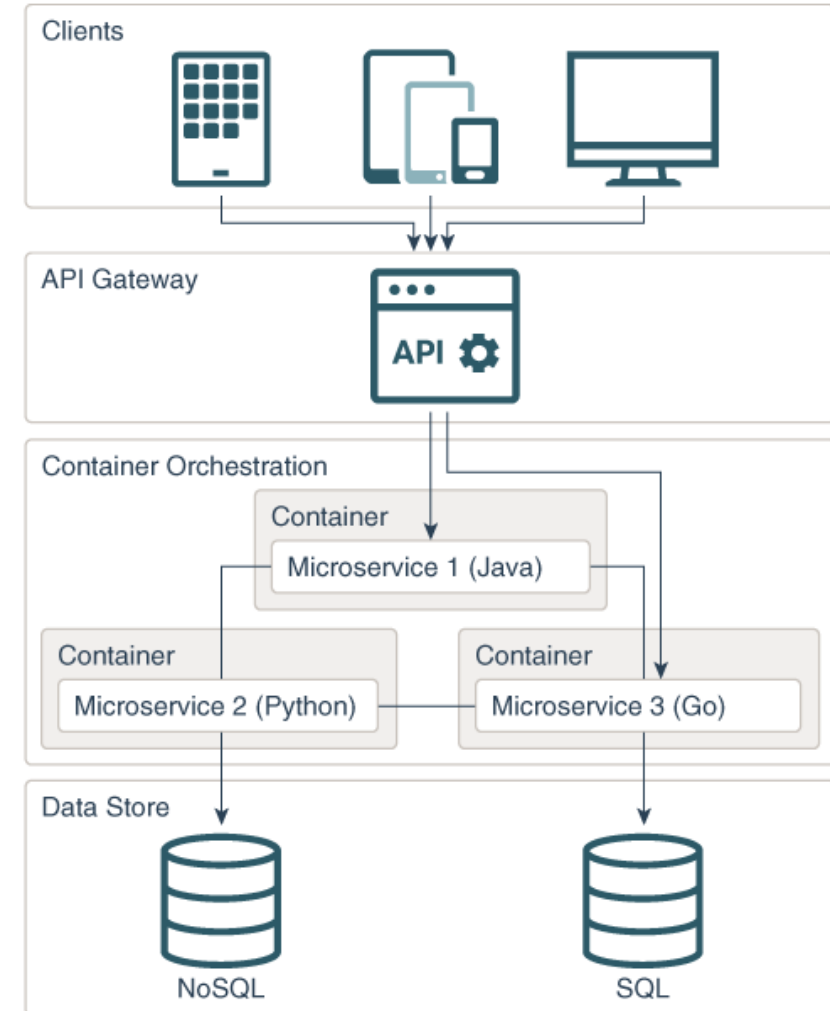
CSA1016 - Software Engineering Seminar CO5 AT2

Presented by

Lithish Kumar R (192424169)

Introduction to Cloud-Native Computing and Microservices

- Cloud-native computing is an approach to building and running applications using cloud infrastructure.
- It uses containers, microservices, automation, and orchestration.
- Microservices break a large application into small, independent services.
- Each service handles one business function and can be developed and deployed separately.

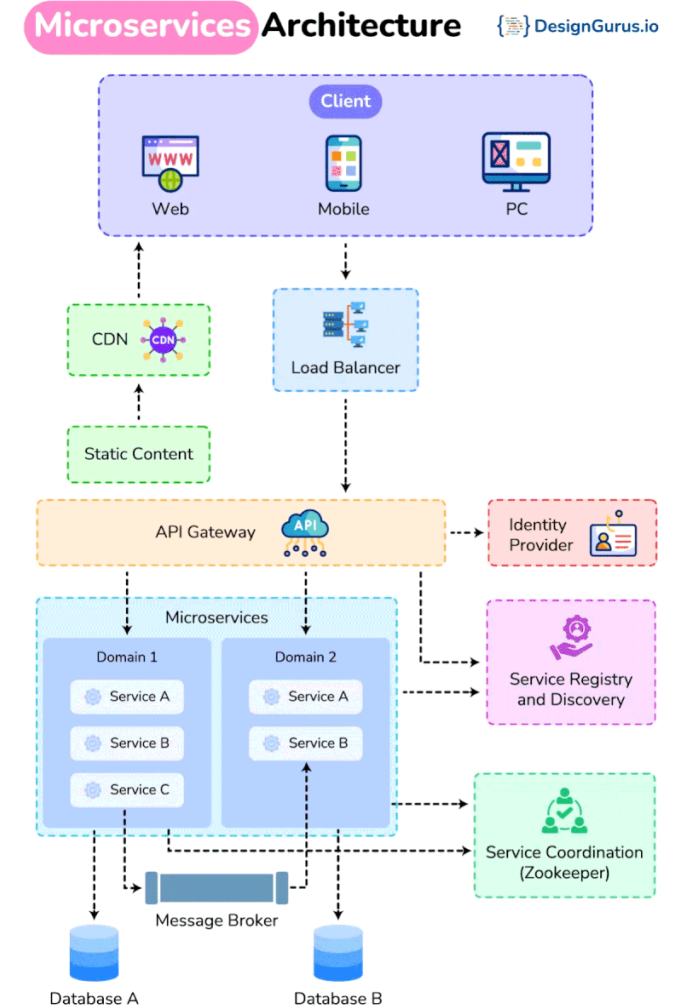


Need and Motivation

- Traditional monolithic applications combine all features—such as user login, payments, database access, and reporting—into one large system.
- When one feature needs an update, the entire application often has to be tested and redeployed. This slows down development.
- As the number of users increases, scaling a monolithic application becomes expensive because the whole system must be scaled, even if only one feature needs more resources.
- A failure in one module can affect the entire application, reducing reliability and availability.
- Organizations today need faster software releases to respond to customer feedback, market changes, and competition.

Working Principle / Architecture

- Users access the application through an API gateway or load balancer.
- Requests are routed to independent microservices.
- Each microservice performs a specific task, such as login, payment, or product search.
- Services communicate through APIs or message queues.
- Containers package services, while Kubernetes manages deployment and scaling.

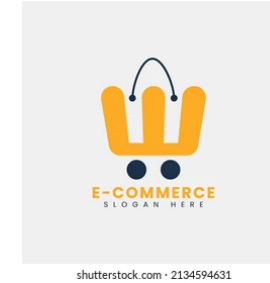


Key Components and Technologies

- Containers: Package applications with their dependencies. Example: Docker.
- Container orchestration: Manages containers at scale. Example: Kubernetes.
- Microservices: Small, independently deployable services.
- API Gateway: Provides a single entry point for clients.
- CI/CD: Automates building, testing, and deployment.
- Monitoring: Tracks application performance. Examples: Prometheus and Grafana

Applications / Real-World Use Cases

- E-commerce: Separate services for catalog, cart, payment, and delivery.
- Streaming platforms: Independent services for recommendations, video delivery, and subscriptions.
- Banking: Secure services for accounts, transactions, and fraud detection.
- Healthcare: Scalable systems for patient records and telemedicine.
- Ride-sharing: Real-time services for location, payments, drivers, and bookings.



Advantages and Limitations

Advantages:

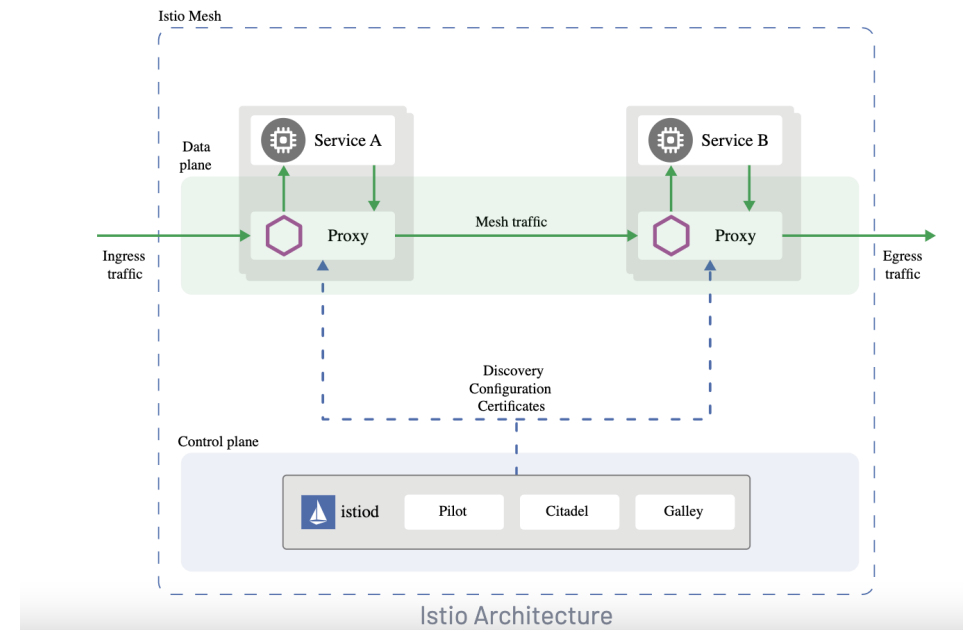
- Faster development and deployment
- Independent scaling of services
- Improved fault isolation
- Better flexibility and technology choice
- Supports continuous delivery

Limitations:

- More complex management
- Network communication can cause latency
- Harder debugging across many services
- Requires strong monitoring and security

Recent Developments and Trends

- Serverless computing is increasingly used with microservices.
- AI-driven monitoring helps detect failures and performance issues.
- Service meshes improve communication, security, and observability.
- GitOps automates infrastructure changes through version control.
- Platform engineering provides developers with reusable cloud tools and environments.



Challenges and Future Scope

Challenges:

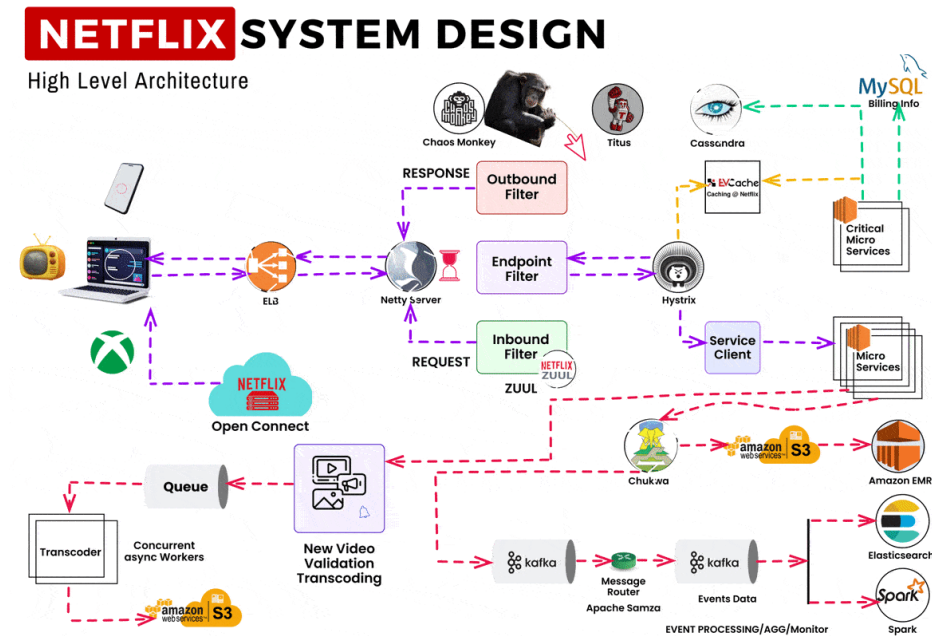
- Managing distributed systems
- Securing APIs and containers
- Controlling cloud costs
- Handling data across multiple services
- Ensuring reliable communication between services

Future scope:

- Greater use of AI for automated operations
- Edge computing for low-latency applications
- More adoption of serverless platforms
- Improved cloud security and zero-trust architectures

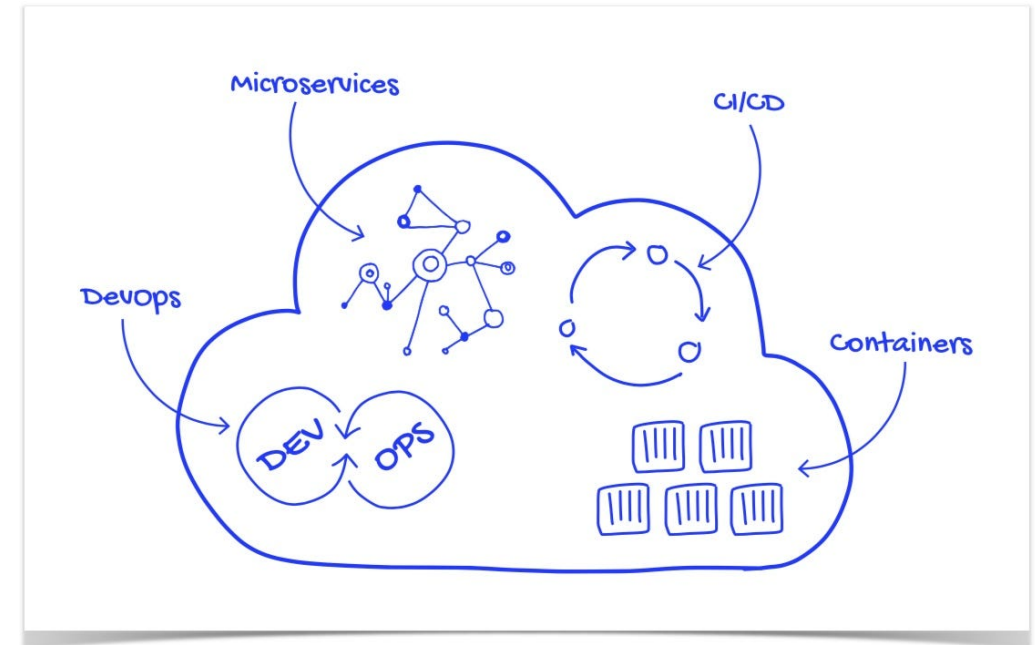
Case Study / Example: Netflix

- Netflix uses a microservices-based architecture to support millions of users.
- Different services manage streaming, recommendations, user profiles, billing, and content delivery.
- Microservices allow Netflix to update features without interrupting the full platform.
- Cloud-native design helps Netflix scale during high-demand periods.



Conclusion

- Cloud-native computing and microservices enable scalable, flexible, and resilient applications.
- They help organizations release software faster and adapt to changing user needs.
- Although they introduce operational complexity, their benefits make them essential for modern software systems.





THANK YOU